

Lecture 09: Inheritance and Polymorphism

Casts, override, and practice with super

Agenda

1. Inheritance in Kotlin (open, override)
2. Upcast, downcast, smart cast
3. Polymorphism
4. Practice: accountant hierarchy and super

A **subclass** specialises a **superclass**: reuses and extends state/behaviour.

- Kotlin classes are **final by default** — mark open to allow subclassing
- Methods/properties to override must be open
- Subclass uses `override` and calls the parent constructor

open class and override

```
1 open class Animal(val name: String) {  
2     open fun speak(): String = "..."  
3 }  
4  
5 class Dog(name: String) : Animal(name) {  
6     override fun speak(): String = "woof"  
7 }  
8  
9 fun main() {  
10     println(Dog("Rex").speak())  
11 }
```

Constructor chaining

- Subclass header must call a parent constructor: `: Animal(name)`
- Parent initialises first, then child property initialisers / `init`
- Next: `super` to call parent methods explicitly

Quiz 1 — open by default?

Quiz

Why does `class Cat : Animal()` fail if `Animal` is declared as `class Animal`?

1. Kotlin forbids inheritance entirely
2. `Animal` is `final` by default; it must be `open`
3. You forgot `public`
4. Subclasses must be `data class`

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

2. `Animal` is final by default; mark it open.

Same for overridable members: they need `open` (unless already abstract).

- **Upcast** — treat a subclass instance as a superclass type (always safe; often implicit)
- **Downcast** — treat a superclass reference as a subclass (may fail at runtime)
- **Smart cast** — after `is` check, compiler narrows the type

Upcast and polymorphism setup

```
1 fun play(a: Animal) {  
2     println(a.speak())  
3 }  
4 fun main() {  
5     val d: Animal = Dog("Rex")    // upcast  
6     play(d)  
7 }
```

Static type is `Animal`; runtime type is `Dog`.

Unsafe vs safe downcast

```
1 val a: Animal = Dog("Rex")
2 val d1 = a as Dog           // throws if wrong
3 val d2 = a as? Dog          // null if wrong
4 if (a is Dog) {
5     println(a.speak())      // smart cast to Dog
6 }
```

Prefer `is` + smart cast or `as?` over blind `as`.

Smart cast limits

Smart casts work when the compiler knows the reference cannot change between check and use (e.g. `val local`).

Common mistake

Smart-casting a `var` property that might be mutated concurrently or via another reference — the compiler may refuse.

Quiz 2 — What happens?

Quiz

Given open class A; class B : A(); val x: A = A(), what does x as B do?

1. Succeeds and returns B
2. Returns null
3. Throws ClassCastException
4. Smart-casts forever

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

3. Throws `ClassCastException`.

Runtime type is A, not B. Use `as?` if failure should be nullable.

Subtype polymorphism: code written against a base type works with any subtype; the *runtime* method body runs (override). Benefits: extend behaviour by adding subclasses without rewriting callers.

Dynamic dispatch

```
1 open class Shape {  
2     open fun area(): Double = 0.0  
3 }  
4 class Circle(val r: Double) : Shape() {  
5     override fun area() = Math.PI * r * r  
6 }  
7 class Square(val side: Double) : Shape() {  
8     override fun area() = side * side  
9 }  
10 fun total(shapes: List<Shape>) =  
11     shapes.sumOf { it.area() }
```

Depend on abstractions (base class / interface), not concrete leaves. Next lectures: abstract classes and interfaces as stronger tools for this.

Quiz 3 — What prints?

Quiz

What prints?

```
1 val s: Shape = Circle(1.0)
2 println(s.area() > 3.0)
```

1. false because Shape.area returns 0.0
2. true (Circle.area runs)
3. Compile error: area not on Shape
4. null

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. true — `Circle.area` runs via dynamic dispatch.

Model employees at a firm:

- `Employee` — `name`, `base salary`, `pay(): Int`
- `Accountant` — employee with a `bonus percent`; overrides `pay`
- Use `super.pay()` to reuse base salary logic

Base type

```
1 open class Employee(  
2     val name: String,  
3     val baseSalary: Int  
4 ) {  
5     open fun pay(): Int = baseSalary  
6     open fun describe(): String =  
7         "$name earns ${pay()}"  
8 }
```

Accountant with `super`

```
1 class Accountant(  
2     name: String,  
3     baseSalary: Int,  
4     val bonusPercent: Int  
5 ) : Employee(name, baseSalary) {  
6     override fun pay(): Int {  
7         val base = super.pay()  
8         return base + base * bonusPercent / 100  
9     }  
10    override fun describe(): String =  
11        "Accountant " + super.describe()  
12 }
```

Polymorphic payroll

```
1 fun main() {  
2     val staff: List<Employee> = listOf(  
3         Employee("Sam", 3000),  
4         Accountant("Alex", 3000, 10)  
5     )  
6     for (e in staff) {  
7         println(e.describe())  
8     }  
9 }
```

Expected idea: Sam gets 3000; Alex gets 3300; describe includes role for accountant.

- Call `super.method()` to extend, not replace entirely
- `super` in constructors: secondary/primary delegation (Lecture 08)
- Prefer small overrides that compose with parent behaviour

Try this

Add `Manager` with a fixed stipend on top of `super.pay()`.

Quiz 4 — super

Quiz

In `Accountant.pay`, why call `super.pay()` instead of reading `baseSalary` only?

1. `super.pay()` is mandatory syntax sugar with no effect
2. It reuses parent policy; if parent `pay` changes, child still composes
3. It performs a downcast
4. It is required to make the class open

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. It reuses parent policy so subclasses compose with base behaviour.

Directly reading fields can duplicate logic and drift when the parent evolves.

Summary

- Mark classes/members `open`; override with `override`
- Upcast is safe; downcast with `as/as?`; prefer `is` smart casts
- Polymorphism: base-typed references, subclass method bodies
- `super` composes child behaviour with parent implementation

Looking ahead

Next: `enum class`, encapsulation / visibility, and `Any / toString`.